How the checker works, step by step

The bonus program: bonus/main.c, bonus/input.c, bonus/operations.c, built with make bonus

The job: take the same numbers as push_swap, read a list of operations from stdin, replay them on the stacks and say whether they sort the input: `OK` if A ends up sorted and B empty, `KO` otherwise, and `Error` for bad numbers or a bad operation. It is the referee for push_swap's output.

1 Arguments

Same join, split and number checks as push_swap, but flags are not allowed.

`check_args_bonus`

2 Read

Read stdin to the end in 42-byte chunks, then split the text into lines.

`build_operations`

3 Validate

Every line must be exactly one of the 11 operation names, before anything runs.

`check_operations`

4 Replay

Build A, then run each operation with print = 0, using push_swap's own functions.

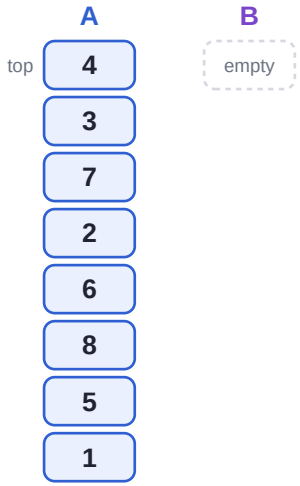
`execute_operations`

5 Verdict

OK if A is sorted and B is empty, otherwise KO.

`is_sorted`

THE EXAMPLE IN THIS WALKTHROUGH



Command `./push_swap 4 3 7 2 6 8 5 1 | ./checker 4 3 7 2 6 8 5 1`. push_swap prints its 19 operations, one per line, and the pipe feeds them to the checker's stdin.

The checker answers **OK**. It never sorts anything itself: it only replays what it is given and inspects the result.

Every table of inputs and outputs in this PDF was produced by running the real binaries.

Step 1: the arguments

bonus/input.c: check_args_bonus and parse_args_bonus, built on push_swap's input code.

What is shared, what is different

`main` starts exactly like push_swap: no arguments means return 0 silently, otherwise `build_args` joins argv and splits it on spaces.

`check_args_bonus` runs the same `check_number` and `check_duplicate` on every word. The difference: if `get_mode` recognises a flag, it is an immediate `Error`. The checker has no strategies and no --bench.

`parse_args_bonus` appends every word to A with `ft_lstnew` and `ft_lstadd_back`, then calls `assign_index`, exactly like push_swap. It runs only after the operations have been read and validated.

The Makefile builds the checker from `bonus/` plus push_swap's `input/`, `operations/`, `libft/` and `algorithms/sort_utils.c` (for `is_sorted`). The sorting algorithms are not linked in.

Examples (with empty stdin)

arguments	output	why
`1 2 3`	`OK`	sorted, no operations needed
`3 2 1`	`KO`	valid, but not sorted
`1 1`	`Error (exit 1)`	duplicate
`2 a`	`Error (exit 1)`	not a number
`2147483648`	`Error (exit 1)`	above INT_MAX
`--simple 2 1`	`Error (exit 1)`	no flags in the checker
`--bench 1`	`Error (exit 1)`	no flags in the checker
`"3 2" 1`	`KO`	quotes work like push_swap
`""`	`OK`	no numbers: an empty A counts as sorted

With no arguments at all (`argc == 1`) the checker prints nothing and does not read stdin.

Step 2: read every operation from stdin

bonus/input.c: build_operations(0) reads to the end of input, then splits on newlines.

THE START OF WHAT ARRIVES ON FD 0 IN THE EXAMPLE

... 61 bytes in total

ft_read

Allocates a buffer of `BUFFER_SIZE` + 1 bytes (42 by default, set in `checker.h`) and loops on `read(fd, buffer, 42)`. Each chunk is NUL-terminated and appended to the text so far with `ft_strjoin`; the old text is freed each time.

The loop stops when read returns 0 (end of input). With a pipe, that is when push_swap exits; typed by hand, it is Ctrl-D.

If read fails (-1) or an allocation fails, everything is freed and `NULL` comes back.

Then ft_split on newlines

The whole text is cut at `\n` into a NULL-terminated array, one operation per entry. If nothing was read, `build_operations` returns `NULL`, which means "no operations".

stdin	lines	result for 3 2 1
`ra\nsa\n`	`ra, sa`	OK
`ra\nsa`	`ra, sa`	OK
`ra\n\n\nsa\n`	`ra, sa`	OK
`(empty)`	`(none)`	KO
`sa\n`	`sa`	KO

Because `ft_split` skips empty pieces, a missing final newline and blank lines in between are both accepted. Empty input is fine too: nothing is replayed, and the verdict is about the input as given.

Step 3: validate before replaying anything

bonus/operations.c: check_operations compares every line with the 11 names.

is_operation

Each line is compared with `ft_strcmp` against pa, pb, sa, sb, ss, ra, rb, rr, rra, rrb and rrr. The match must be exact: no spaces, no other case, nothing extra.

The first line that is not an operation ends the program: the operation array is freed, then `print_error` frees the arguments and writes `Error` to stderr with exit code 1.

All lines are checked before the first one runs, and before stack A is even built. So an invalid line at the very end still means no replay at all, never a half-finished one.

stdin	output for 3 2 1	why
`ra\nsa\n`	`OK`	two valid operations
`ra \nsa\n`	`Error (exit 1)`	trailing space
`RA\nsa\n`	`Error (exit 1)`	upper case
`ra\nsa\nfoo\n`	`Error (exit 1)`	unknown, even at the end
`rra\n`	`KO`	valid, but leaves A unsorted
` ra\n`	`Error (exit 1)`	leading space
`ra\tsa\n`	`Error (exit 1)`	tab is not a separator

THE 11 NAMES

`sa` `sb` `ss` `pa` `pb` `ra` `rb` `rr` `rra` `rrb` `rrr`

Step 4: replay the operations

bonus/operations.c: execute_operations calls push_swap's own sa, pb, ra ... with print = 0.

`init_stacks` and `parse_args_bonus` build A, then each line goes through `execute_operation`, an if-chain from name to function. With `print = 0` nothing is written, but the stacks move exactly as they did inside push_swap. Below: the example, one row per line of stdin.

#	op	A after (top first)	B after (top first)
1	pb	3 7 2 6 8 5 1	4
2	pb	7 2 6 8 5 1	3 4
3	rb	7 2 6 8 5 1	4 3
4	ra	2 6 8 5 1 7	4 3
5	pb	6 8 5 1 7	2 4 3
6	rb	6 8 5 1 7	4 3 2
7	rra	7 6 8 5 1	4 3 2
8	rra	1 7 6 8 5	4 3 2
9	pb	7 6 8 5	1 4 3 2
10	rra	5 7 6 8	1 4 3 2
11	pb	7 6 8	5 1 4 3 2
12	sa	6 7 8	5 1 4 3 2
13	pa	5 6 7 8	1 4 3 2
14	pa	1 5 6 7 8	4 3 2
15	ra	5 6 7 8 1	4 3 2
16	pa	4 5 6 7 8 1	3 2
17	pa	3 4 5 6 7 8 1	2
18	pa	2 3 4 5 6 7 8 1	-
19	rra	1 2 3 4 5 6 7 8	-

After line 19, A reads 1 2 3 4 5 6 7 8 and B is empty. A move that has nothing to do, such as pa with B empty, is simply a no-op here, just as in push_swap.

Step 5: OK or KO

bonus/main.c: one condition decides, then everything is freed.

```
if (is_sorted(stacks.a) && !stacks.b)
    ft_putendl_fd("OK", 1);
else
    ft_putendl_fd("KO", 1);
```

`is_sorted` (algorithms/sort_utils.c) walks A and checks that the indexes never go down. An empty A counts as sorted.

`!stacks.b` demands that B is empty: every number must be back in A. A sorted A with something left in B is still KO.

Then `free_args` frees the words and the operation lines, `free_stacks` frees the nodes, and main returns 0. OK and KO both exit with 0; only Error exits with 1.

arguments	stdin	output	why
`4 3 7 2 6 8 5 1`	`its 19 lines`	`OK`	push_swap's output
`3 2 1`	`ra\nsa\n`	`OK`	sorts it
`3 2 1`	`sa\n`	`KO`	A is 2 3 1
`1 2 3`	`(empty)`	`OK`	already sorted
`3 2 1`	`(empty)`	`KO`	nothing done
`1 2 3`	`pb\n`	`KO`	A sorted, B not empty
`3 2 1`	`3x pb, 3x pa`	`KO`	out and back: 3 2 1
`1 2`	`sa\nsa\n`	`OK`	two swaps cancel out

Quick reference

Building, running and testing with the checker.

Build and run

```
make bonus                      # builds ./checker

ARG="4 3 7 2 6 8 5 1"
./push_swap $ARG | ./checker $ARG # OK
printf "ra\nsa\n" | ./checker 3 2 1 # OK
./checker 3 2 1                 # type, then Ctrl-D
```

Testing push_swap with random input

```
for i in $(seq 50); do
     shuf -i 0-9999 -n 100 > args.txt
     ./push_swap $(cat args.txt) | ./checker $(cat args.txt)
     ./push_swap $(cat args.txt) | wc -l
done
```

Outputs

output	stream	exit	when
`OK`	stdout	0	A sorted and B empty after the replay
`KO`	stdout	0	anything else
`Error`	stderr	1	bad number, duplicate, flag, or bad line
`(nothing)`	-	0	no arguments

Files

file	contains
`bonus/checker.h`	BUFFER_SIZE (42) and the 5 bonus prototypes
`bonus/main.c`	the five steps and the verdict
`bonus/input.c`	argument checks, A, reading stdin
`bonus/operations.c`	validating and running the lines